

WaveVis inverse-sheet architecture for rounded breaking-wave linkage arrays

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet record

WaveVis is an inverse computational-design record for a rectangular linkage sheet that is being driven toward one localized plunging-wave deformation while keeping the mechanism visible. This PDF is not allowed to be a decorative paper, a candidate gallery, or a misleading overlay. It is the reconstruction manual for the project: the target references, coordinate frames, source files, equations, topology constraints, render/display split, validation gates, failure ledger, deployment boundary, and next iteration protocol needed for a fresh engineer to continue from source truth. The accepted reference family is now limited to the June 24 smooth gridded overview/sequence, the June 28 stage/cross-section sheet, the July 1 supplied side-view curl line used only for the side-view trace, and the retained ocean-curl photographs used only as secondary water-form references. The earlier trace-against-silhouette overlay is rejected paper evidence: it made the target look too pointy, encouraged profile-only fitting, and must not appear as an authoritative figure. The July 1 supplied side-view line is accepted as a side-view silhouette source, not as the full three-dimensional sheet target. The rejected geometry family is equally explicit: swept tunnels, side walls, skinny center-strip sails, triangular cuts, pointy side profiles, bowls, dimples, dark pockets, under-overhang support necks, profile-only completion claims, hidden mechanisms, fake filler bridges, old candidate screenshots, wrong or irrelevant stills, and any figure that makes the wrong geometry look authoritative. Current state is not a reference-match publication state. Candidate876 remains the last committed and previously route-checked baseline, and Local Candidates 898 through 903 remain rejected diagnostics because they either kept a pointy/spiral side read, became a blunt cylinder or half-pipe, collapsed into a mound without a curl, lost the overhang, or became a broad wall/blob. The July 1, 2026 repair after the pointy-profile complaint split the readable display target into a smoothed image-derived side-only reference trace and a separate localized three-dimensional/isometric trace, added numeric blockers for pointy/base-turn side traces, and widened the side camera framing for desktop/mobile inspection.

29 **That repair removes the old spear-like side read from the local source, but it does not close the**
30 **full reference-match problem. The next valid iteration must preserve this side-profile blocker**
31 **while improving the rendered full mechanism against the smooth gridded reference family, not a**
32 **profile-only surrogate.**

33 **Fresh-chat use.** Read this PDF before changing WaveVis. Then inspect the current source, the live
34 route, the latest handoff, and the rendered artifact. If any of those disagree, the live/source/rendered
35 state wins over this prose until the PDF is rebuilt and reverified. A future chat should be able to
36 reconstruct the model and the iteration state from this record without relying on old chat memory.

Contents

37

1	Introduction	6	38
1.1	Reference geometry	7	39
2	Start here for a new WaveVis chat	12	40
2.1	What the next agent must not repeat	15	41
2.2	Current state ledger	16	42
2.3	Figure and asset hygiene	17	43
2.4	File map	19	44
2.5	Minimal continuation workflow	20	45
3	Current implementation state	20	46
3.1	Defaults and target	21	47
3.2	Target-field construction	21	48
3.3	Reconstruction-grade mathematical model	22	49
3.3.1	Custom profile and readable-view split	23	50
3.3.2	Generated lip path	25	51
3.3.3	Span localization	26	52
3.3.4	Bowl/dimple blocker	27	53
3.3.5	Connected X-cell equations	27	54
3.3.6	Acceptance objective	28	55
3.3.7	Candidate iteration record	29	56
3.4	Rigid controls	29	57
3.5	Morph control	30	58
3.6	Flat contribution and ground transition	30	59
3.7	Lip dip and terminal curl	30	60

61	3.8 Rendering contract	31
62	4 Target outcome	32
63	4.1 Human-facing shape	32
64	4.2 Hard constraints	33
65	5 Coordinate model	33
66	6 Generation pipeline	34
67	7 Supplied rounded target	35
68	8 Slider semantics	36
69	9 Breaking-wave geometry	36
70	10 Mechanism topology	37
71	11 Rendering modes	38
72	11.1 Magnetic valve sandbox	39
73	12 Verification gates	39
74	12.1 Current regression command set	41
75	12.2 Rendered verification	43
76	13 Build, deploy, and publication runbook	43
77	13.1 Local development	43
78	13.2 Public fallback deployment	44
79	13.3 Custom-domain boundary	44
80	13.4 Progress logging	44
81	14 Current verified and open state	45

15 Failure modes	47	82
16 Materials and Methods	48	83
17 Discussion	49	84
18 Acknowledgements	49	85
19 Funding	50	86
20 Author contributions	50	87
21 Competing interests	50	88
22 Data availability	50	89
23 Code availability	50	90
24 Additional information	50	91
25 References	51	92

1 Introduction

WaveVis is an inverse-design simulator for a deployable mechanism. It takes a target surface intent, samples it on a rectangular sheet, and renders the resulting cell/connector mechanism rather than only a smooth mesh. That places it between several technical families: deformable-body, cloth, shell, rod, and mass-spring simulation where surfaces and slender members are discretized, regularized, and advanced under mechanical constraints (1–5); spacetime, constraint-projection, projective-dynamics, and differentiable-simulation methods where feasibility is enforced by explicit objective terms, local/global corrections, or gradients (6–10); computational mechanism design where actuator placement, linkage topology, compliant flexures, and contact constraints are part of the design object (11–17); and inverse/topology design of mechanisms, deformation-programmed materials, inflatables, auxetics, soft robots, and metamaterials where a desired deformation field must be made compatible with a manufacturable structure (18–27). WaveVis is not a solved member of any of those literatures. It is an active reconstruction record for one specific mechanism question: can a rectangular linkage sheet be made to read as a localized plunging wave while keeping its full kinematic graph visible?

The design problem is stricter than drawing a curve or matching a screenshot. A side outline can look acceptable while the actual three-dimensional sheet has a wall across the underside, a cavity where the lip should be open, an extruded barrel, or zig-zagging linkage legs that no longer read as a mechanism. Conversely, a mechanism can satisfy a local numerical check while failing the human outcome: a full rectangular sheet with a localized plunging wave. WaveVis must therefore treat the visible artifact, target field, kinematic topology, URL state, source checks, paper figures, and public deployment as one coupled system.

The central correction captured here is that a breaking wave is not made by sweeping one two-dimensional profile sideways. A swept profile creates a tunnel, canopy, or half-pipe. The required object is a localized deformation field on a flat sheet:

$$\Phi : (x, y) \mapsto (x + \Delta_x(x, y), y + \Delta_y(x, y), \Delta_z(x, y)),$$

where the displacement is concentrated near the middle span, gradually fades toward the lateral sides, and is exactly zero on the perimeter. The curl must be strongest near the centerline and weaker toward the sides. This spatial falloff is part of the target shape, not a rendering trick.

The hard failure classes are not subtle. One rendered state showed an inward support neck and dimple below the curling lip. Another rendered side profile ended in a pointy surrogate curl that did not match the July 1 supplied side-view reference. Both classes are banned because they make the simulator solve the wrong object: either a local support column where the reference requires an open throat, or a side-view trace whose crest, hook, and low return are wrong. No overlay, centerline plot, paper caption, or metric-only pass may convert those failures into evidence.

1.1 Reference geometry

The design reference is the gridded breaking-wave reference set, the July 1 supplied side-view curl line, and retained ocean-curl photographs: a smooth, localized volume that rises out of a larger flat sheet, grows into a broad rounded crest, curls into a smaller inward spiral, and finishes with a smooth downturned tapered lip while leaving an open underside and a long low return. The July 1 side line is accepted only as the side-view silhouette source; it cannot define the full three-dimensional sheet, top-view footprint, front/cross-section body, or mechanism visibility contract by itself. The older trace-against-silhouette overlay is rejected because it made the target look pointy and mixed an internal centerline diagnostic with a supposed paper reference. These reference frames are not decorative. They are the visual contract for the simulator's target geometry. A successful WaveVis state should read as a simplified linkage approximation of this water form: the outer radius is large and smooth, the inner curl radius is smaller, the lip projects forward before curling down, the front/cross-section view is rounded rather than pointed, and the side field fades away instead of forming a constant barrel. This is deliberately stricter than matching any side-profile curve: the full linkage array in side and isometric views must preserve the open curl without side walls, erratic zig-zags, disconnected cells, or hidden filler geometry.

Current verification boundary. This document records the target, architecture, and required gates. It does not by itself prove that the current generator has matched the supplied references. The rendered full-array artifact remains the source of truth. If the page, screenshots, or live route do not visibly match the localized plunging-wave reference family while preserving linkage cells and flat perimeter, the correct state is “open visual mismatch,” not “reference matched.”



Figure 1. Primary supplied reference geometry. The target form is a localized plunging water sheet with a rounded crest, open underside, and forward/downward lip. The WaveVis sheet must approximate this geometry while preserving flat perimeter boundary nodes and visible linkage cells.



Figure 2. Secondary supplied open-throat reference. The image is used only to define the underside condition: the lip is suspended, the throat remains open, and no side wall, support neck, or dark pocket may be used to close the curl.

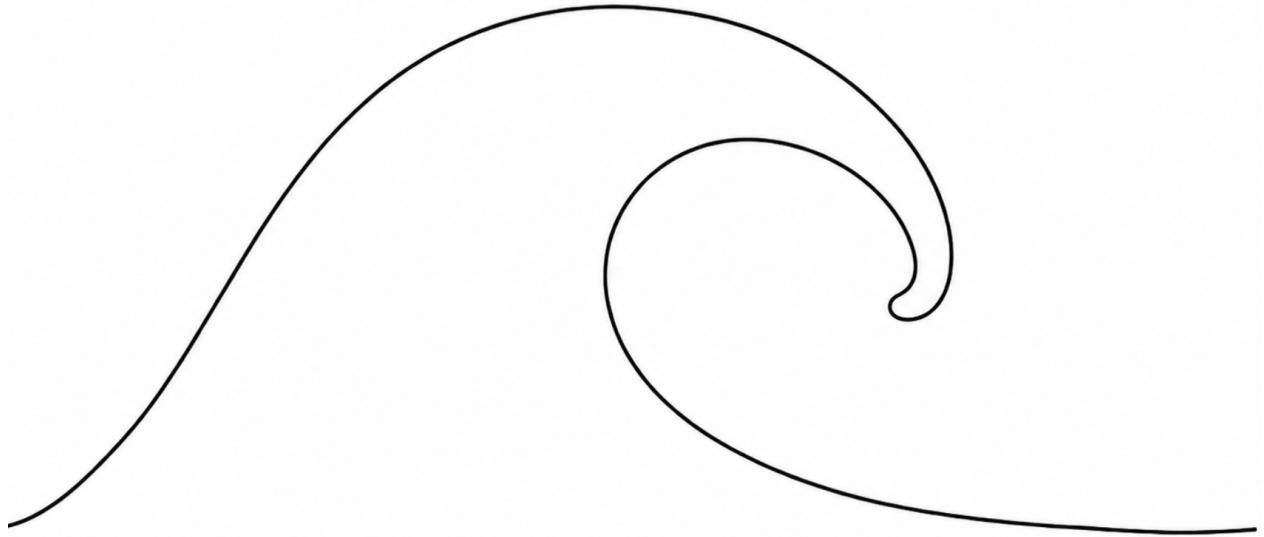


Figure 3. July 1 supplied side-view curl reference. This image is the source for the readable side-only trace in `src/LatticeViewer3D.tsx`. It is not a full three-dimensional mechanism target by itself: top, front, isometric, open-throat, perimeter-flatness, and mechanism-visibility gates still decide whether the rendered WaveVis sheet is acceptable.

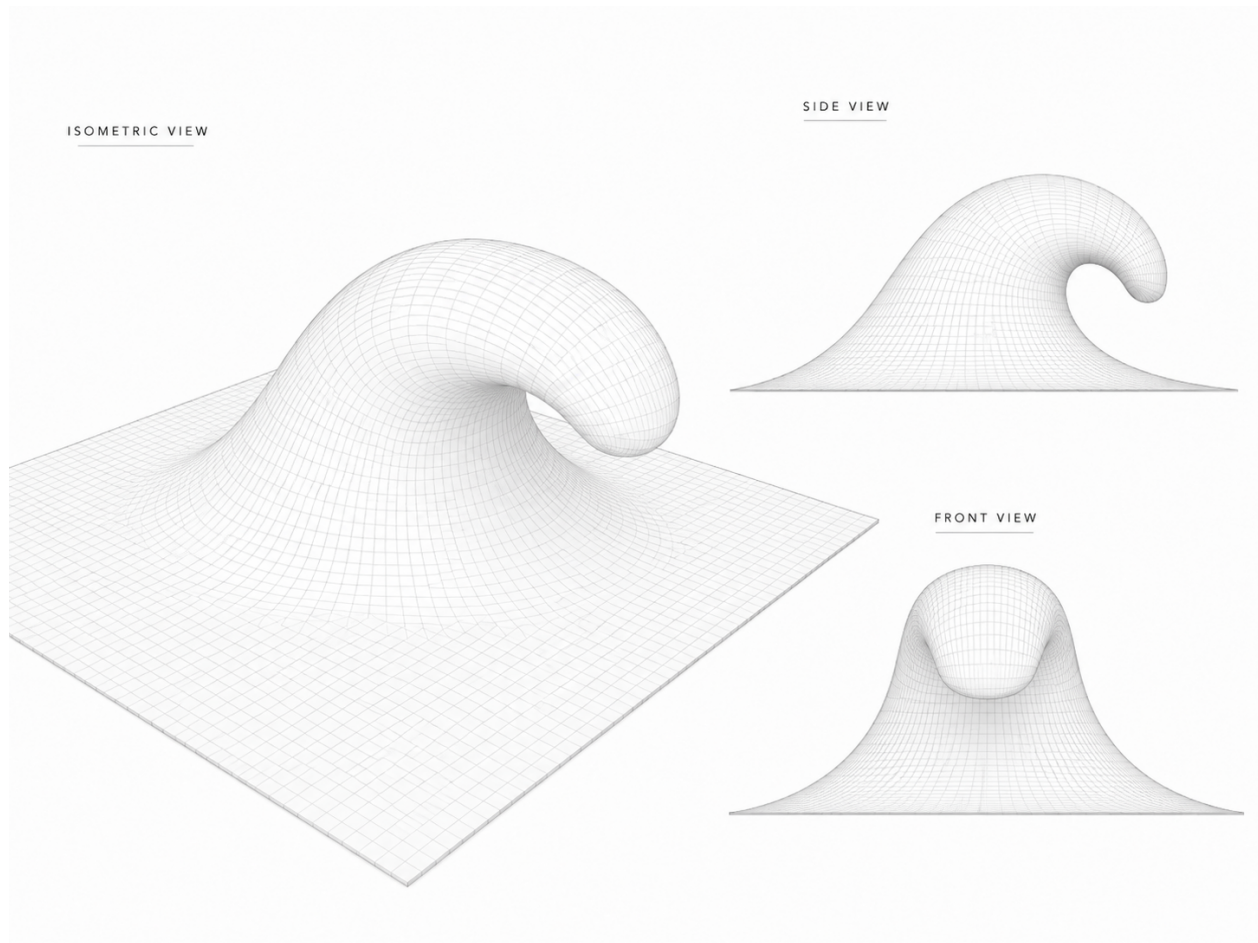


Figure 4. June 24 supplied smooth gridded reference overview. Alan re-supplied this image and said “remember.” It is now a preserved project reference for the desired visual grammar: a continuous square sheet surface, rounded rising face, tube-like crest, downturned lip, open clean throat, and centered front view. A jagged linkage trace, side-wall tunnel, or metric-only improvement does not satisfy this reference.

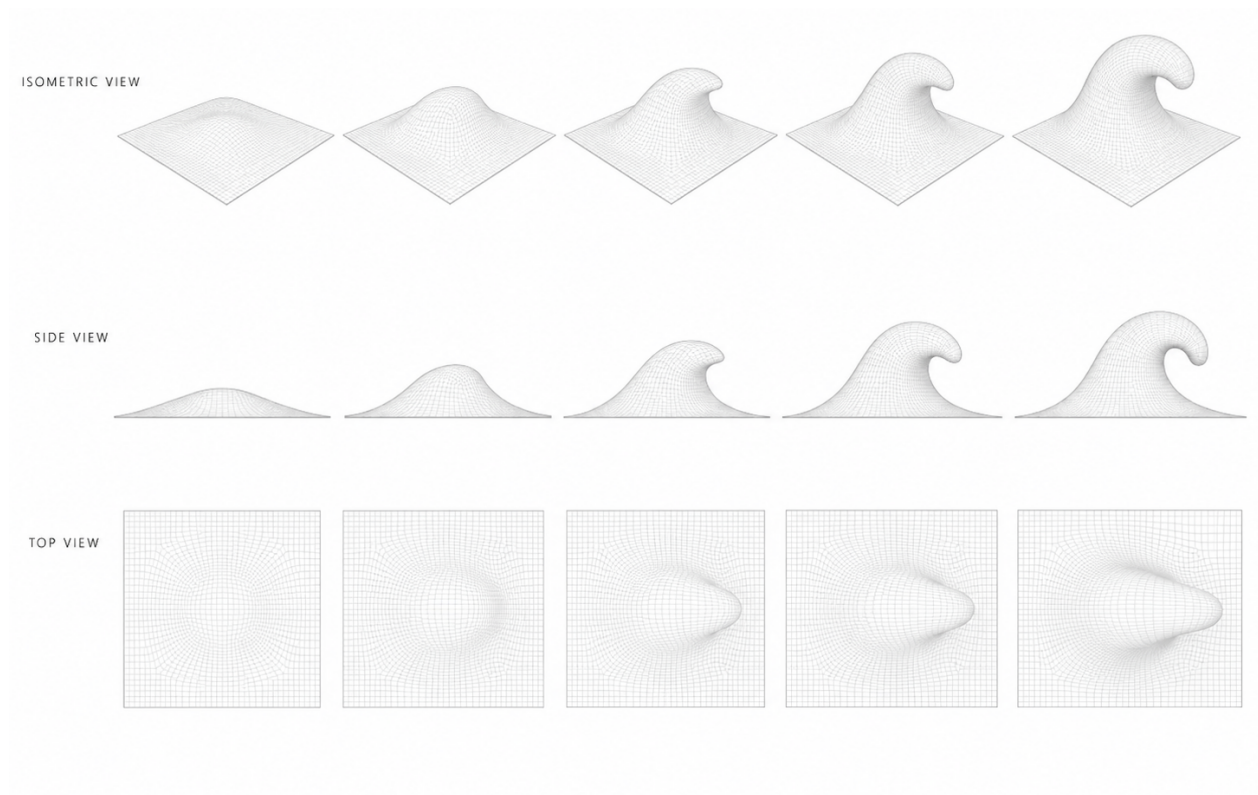


Figure 5. June 24 supplied smooth gridded reference sequence. The sequence records the intended progression across isometric, side, and top views: the deformation grows from a flat square sheet into a localized rounded wave with a curled lip while the top view remains a rounded localized body, not a triangular fan or swept strip.

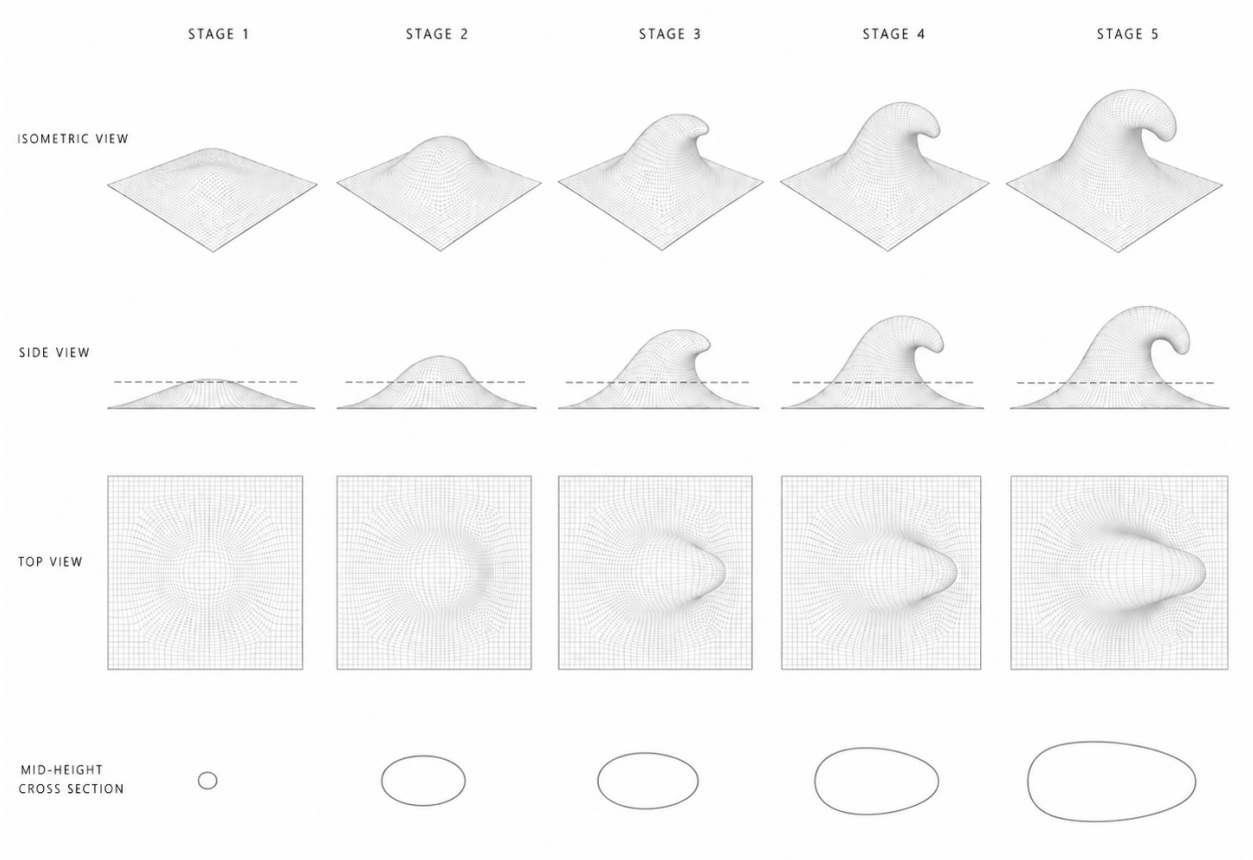


Figure 6. June 28 supplied stage sequence with an explicit mid-height cross-section row. The bottom row tightens the plan-footprint target: the active body should grow into one smooth oval or teardrop-like section through the curl, not a pointed spear, pinched hourglass, terminal stripe, or pasted helper contour. This reference is now part of the WaveVis acceptance set.

2 Start here for a new WaveVis chat

If this PDF is the only WaveVis context available, start with this section and then inspect the live artifact. The current project state is:

- **Verified public route:** <https://aolabs.io/wavevis/>. This route serves the fallback bundle that Alan actually uses.
- **Working standalone route:** <http://wavevis.aolabs.io/>. This route serves the current WaveVis bundle. The HTTPS version remains a separate certificate boundary.
- **Preferred HTTPS route still blocked:** <https://wavevis.aolabs.io/>. At the time of this record, HTTPS validation can fail because the custom-domain certificate is not valid for that

hostname. Do not treat the HTTPS subdomain as the verified route until a fresh certificate check passes.

- **Visible PDF action:** the WaveVis header links to the paper file under `/proofs/`. File: `wavevis-system-architecture.pdf`. The old connector proof remains a source artifact, not the primary user-facing paper action.
- **Current source root:** workspace path `_apps/wavevis.aolabs.io/`.
- **Public fallback mirror:** workspace path `_apps/aolabs-site/wavevis/`. A WaveVis deploy to its own `gh-pages` branch does not automatically update the `aolabs.io/wavevis/` fallback unless the built files are mirrored there and the hub site is pushed.
- **Current architectural direction:** curated supplied-reference field localized into the inverse-sheet model, plus a separate readable display surface for visual inspection. Manual *xz* and *xy* profile editors are not the active path.
- **Current branch state:** Candidate876 is the last committed baseline source state, but the working July 1 side-profile repair changes `src/LatticeViewer3D.tsx`, `src/latticeGeometry.ts`, and `scripts/check-inverse-sheet.cjs`. The repair replaces the pointy readable side trace with a smoothed side-only trace extracted from Figure 3, keeps top/isometric/front on a localized three-dimensional reference field, releases the suspended lip back to the flat body before the endpoint, weakens the top-view shadow so it no longer reads as a dark rectangular wall, widens the side camera framing, and adds source-level rejections for side traces whose terminal backtrack ratio is too small or whose low backtracked turn collapses near the flat base. Candidate841 through Candidate875 are rejected or superseded diagnostics that explain support-neck dimples, broad roof/curtain, triangular cut/fin, side-wall tunnel, pointy side profile, full-length front projection, and squared terminal foot failures. Candidate898 through Candidate903 are newer rejected local diagnostics: old smooth trace kept a spiral/pointy side and pinched top; tail release became a blunt cylinder; smooth product profile became a loop/half-pipe; no-loop cap became a mound with no curl; Moana lip anchors made a steep nose without the needed overhang; analytic rounded curl became a broad wall/blob.
- **Current verification state:** `npm run check:geometry` passes and `npm run build` passes on the July 1 side-profile repair. The inverse-sheet checker reports these named contracts as true:


```
readableSurfaceRenderContract.ok=true
```

```
186         readableSideProfileCavityBlock.ok=true
187         readableTunnelSweepBlock.ok=true
188         xRenderDirectLines.sourceUsesConnectedXMechanism=true
```

189 The side trace pointy-notch blocker is active: `terminalBacktrackRatio=0.2658`, above the
190 rejection floor `0.24`. The current side trace reports:

```
191         pointCount=72
192         crest=(0.505, 1)
193         terminalBacktrack=0.281
194         terminalFlatRunFromEightPercent=0.31
195         minLowReturnX=0.482
196         maxAbruptPostCrestAngleDeg=7.6085
197         maxTerminalDescentAngleDeg=81.0851
```

198 Browser screenshots exist in:

```
199         _verification/
200         browser-20260701-side-reference-smoothed/
201         browser-20260701-side-long-tail/
202         browser-20260701-side-rounded-arc/
```

203 These checks prove that the July 1 repair is buildable, has a closed source-level mechanism checker,
204 and is no longer the earlier pointy side-profile candidate. They do not prove exact reference match
205 or live deployment by themselves. The preferred HTTPS subdomain remains certificate-blocked
206 until a fresh certificate check passes.

- 207 • **Correct acceptance target:** the supplied June 24 smooth breaking-wave reference family, the June
208 28 stage/cross-section reference, and the July 1 side-view reference for the side-only silhouette. The
209 wrong target is the removed internal trace-overlay diagram, a centerline-only plot, a profile-only
210 completion claim, or any helper contour that cannot reconstruct the mechanism.
- 211 • **Current mechanism contract:** every interior cell is one black square body with four straight legs;
212 opposite leg pairs are equal length and collinear through the cell center; every spherical connector
213 node is a physical two-cell joint with exactly two attached leg uses; crossing diagonal families are
214 separate joints rather than four-cell connector collisions.
- 215 • **Do-not-claim boundary:** do not claim reference match, paper current, or public completion from
216 a geometry log, a compiled PDF, a local screenshot, an internal overlay, or a smoother profile string.

The rendered full mechanism is the acceptance object.

217

2.1 What the next agent must not repeat

218

The repeated failures that this document is meant to prevent are:

219

1. Do not make a swept tunnel, ribbon, canopy, roof, half-pipe, bridge, or extruded strip. 220
2. Do not form a bowl, dimple, cavity, pucker, under-overhang throat bite, or side wall that covers or pinches the underside of the curl. 221
222
3. Do not replace Side with a profile-only debug view. Side is a side camera view of the full three-dimensional sheet. 223
224
4. Do not hide, ghost, clip, or remove linkage cells because they look bad. Fix the geometry. 225
5. Do not remove the full array, one-center X cells, exact collinear equal opposite pairs, two-use physical connector joints, or two-cell linkage truth while chasing a prettier outline. 226
227
6. Do not wire the lip directly to the ground or draw filler geometry across the open underside. 228
7. Do not make position or steer shape controls. They are rigid post-shape transforms. 229
8. Do not call the work done from code intent, automated checks, a nicer profile line, or a local-only screenshot. 230
231
9. Do not let the top view collapse into a big triangle, wedge, or V fan while the side view looks acceptable. 232
233
10. Do not paste a mid-height contour, helper ellipse, bridge, or cross-section overlay into the product view to fake the June 28 cross-section row. Fix the surface projection and preserve the X-only proof path. 234
235
236

2.2 Current state ledger

This subsection replaces the old Candidate785 figure gallery. Those screenshots are historical diagnostics, not final paper evidence. The paper must not show a stale candidate as if it were the correct geometry. The current ledger is:

Class	Current source	Paper/display rule
Accepted references	June 24 smooth gridded overview/sequence, June 28 stage/cross-section reference, July 1 side-view curl line for side-only tracing, and retained ocean-curl photos used only for water-form context.	Keep as target figures and cite them as references, not as achieved WaveVis output. The July 1 side line constrains only the side silhouette; it does not replace top/front/isometric or mechanism gates.
Rejected display asset	Profile-trace overlay image, old pointy side-profile surrogate, and non-target stills.	Removed from the paper figure sequence; do not use internal trace overlays, side-line completion claims, or visually irrelevant stills as paper figures.
Rejected candidate states	Candidate785, Candidate841 through Candidate875, and Candidate898 through Candidate903 screenshots.	Historical diagnostics only. Do not describe them as accepted renders after Alan identified support-neck, pointy-profile, front-fin, wall, blob, cone, terminal-foot, blunt-cylinder, half-pipe, no-curl mound, steep-nose, and broad-wall families.
Current source repair	July 1 side-profile repair after the pointy-profile complaint: smoothed image-derived side trace, separate localized 3D/isometric trace, endpoint release, softened top-view shadow, side camera fit, terminal-backtrack floor, low-turn-height blocker, and side-reference descent limit.	Treat as the current source state after <code>npm run check:geometry</code> and <code>npm run build</code> pass. It removes the prior pointy side profile but remains an open visual-match state until the live rendered full mechanism is verified against the accepted references.
Mechanism invariant	One-center four-leg X cells with exact opposite-pair collinearity/equal length and physical two-use connector nodes.	Keep as a hard gate and cite source/checker values after each accepted candidate.
Open boundary	Exact rounded reference-family match, public/fallback route parity, and PDF figure/source consistency.	Blocks final reference-match claims until live routes, screenshot views, and PDF renders are verified.

Table 1. WaveVis state ledger as of July 1, 2026. The paper tracks which information is reference truth, rejected history, active source state, and still-open defect state.

The only current figures that belong in the paper by default are target references and mechanism-proof figures that still define the actual constraint. Candidate screenshots belong only when they are labeled as rejected or diagnostic and they help the next iteration avoid a known failure.

2.3 Figure and asset hygiene

WaveVis now treats paper figures as source evidence, not decoration. A file can appear in this PDF only if it belongs to one of four classes:

1. a target reference that defines the accepted geometry family;
2. a current accepted render that passed source checks, browser inspection, and route verification;
3. a mechanism proof figure that defines a hard constraint still used by the source;
4. a clearly labeled rejected diagnostic that explains a failure mode future iterations must avoid.

Profile-trace overlays, centerline-only comparisons, stale candidate screenshots, unrelated stills, helper contours, and any image whose purpose is to make the current source look more correct than the rendered mechanism must stay out of the paper and the public proof folder. When an asset is removed for this reason, the removal is part of the source-of-truth record rather than cleanup churn.

2.4 File map

File or route	Ownership
<code>src/latticeGeometry.ts</code>	Source of truth for the inverse sheet model, defaults, sanitization, target displacement field, morph behavior, metrics, and geometry sanity checks.
<code>src/LatticeViewer3D.tsx</code>	Source of truth for Canvas rendering, side/isometric/top full-sheet camera behavior, surface visibility, camera presets, visual layers, and the rendered collinear equal-pair X-cell and shared-joint layers.
<code>src/xCellMechanism.ts</code>	Source of truth for the connected X-cell display mechanism: one black-square cell body per lattice cell, four straight center-to-connector legs per cell, exact collinear/equal opposite pairs through the cell center, and spherical connector nodes with exactly two attached leg uses.
<code>src/TargetShapeControls.tsx</code>	Source of truth for the visible inverse-sheet controls: rows, columns, views, scale, morph, position, steer, display toggles, reset, export, and import.
<code>src/InverseSheetTab.tsx</code>	Source of truth for URL-state loading, tab order, view requests, import/export plumbing, and the Inverse Sheet first-tab behavior.
<code>src/MagneticValveTab.tsx</code>	Source of truth for the magnetic-valve sandbox view: tube, ball, barbed fitting seats, magnet and steel placement, coil wrapping, pulse controls, flow direction, pressure controls, and visible cutoff/readout behavior.
<code>src/magneticValveModel.ts</code>	Source of truth for the lightweight magnet-pneumatic valve model. It is a client-side design-intuition model for build selection, not a finite-element, COMSOL, measured magnetic-force, or validated flow solver.
<code>scripts/check-inverse-sheet.cjs</code>	Main inverse-sheet regression checker. It compiles the TypeScript geometry and checks parameter semantics, lip curl, flatContribution, low-lip stability, lateral smoothness, no bowl pockets, terminal taper, startup URL coverage, and the collinear equal-pair X-cell render contract.
<code>scripts/check-magnetic-valve.cjs</code>	Magnetic-valve regression checker for flow cutoff, active-seat reversal, pressure scaling, tube/orifice resistance, magnet placement bias, and Alnico pulse/winding direction.
<code>scripts/check-geometry-invariants.cjs</code>	Mechanism-level invariant checker for rigid-cell/linkage behavior.
<code>proofs/wavevis-system-architecture.tex</code>	Source for this PDF. Update it when the public simulator contract, current state, or known failure modes change.
<code>proofs/figures/</code>	Project-defining reference images and mechanism proof figures used by this paper. Old current-state screenshots and trace overlays must stay out of the paper unless explicitly labeled as rejected diagnostics.
<code>dist/</code>	Built WaveVis static app after <code>npm run build</code> . Do not edit by hand.
<code>aolabs-site/wavevis/</code>	Public fallback mirror for https://aolabs.io/wavevis/ . Copy the built dist contents here when the public fallback must update.

Table 2. Current source map. A new WaveVis chat should use these files rather than reconstructing behavior from old prompts.

2.5 Minimal continuation workflow

A fresh agent continuing WaveVis should do the following in order:

1. Read this PDF and `src/latticeGeometry.ts`, then inspect `src/LatticeViewer3D.tsx` and `src/TargetShapeControls.tsx`.
2. Open the verified public route `https://aolabs.io/wavevis/` and the current local route if a dev server is running.
3. Reproduce Alan's active preset from the URL before changing source. Do not judge a different default state.
4. Run `npm run check:geometry`. If it fails, fix source before visual tuning.
5. Render and inspect Side, 3D, and Top. Side and 3D must both be full-sheet views with the curl visible.
6. Make the smallest source change that improves the human-facing breaking-wave outcome without regressing the mechanism invariants.
7. Re-run `npm run check:geometry` and `npm run build`.
8. Verify live/public output after deploy or fallback mirror update. A local screenshot is not the public artifact.
9. Update this PDF if the architecture, current state, source map, controls, or known failure classes changed.
10. Post a Progress event with complaint, issue, Codex change, observed source change, provenance, and commit ids.

3 Current implementation state

3.1 Defaults and target

The current inverse-sheet default is intentionally taller and sharper than earlier flat presets:

```

        rows = 44,                columns = 112,                morph = 1,
        height = 15.25, horizontalOffset = 22.5,    overhangWidth = 32,
overhangAngleDeg = 120,    overhangPosition = -0.15,    profileScale = 0.60,
        smoothing = 1,            lipSharpness = 0.50,    wallSmoothness = 0.28,
        flatContribution = 0.35.

```

The app displays only the narrow active controls by default. Legacy inputs such as `height`, `overhang`, `lipDip`, `width`, `lipSharpness`, `groundTransition`, `wallSmoothness`, and `flatContribution` can still be parsed from URLs and JSON imports because older test links and screenshots use them. The visible startup path favors `scale`, `morph`, `position`, and `steer` to keep the main wave profile stable.

The default reference-wave sheet remains `columns = 112`. The sanitizer now allows `columns ≥ 12` so X-cell proof routes can be inspected at low density, but compact column counts are not evidence of an exact reference-wave match. Dense-wave regression checks request the default column count explicitly; low-column screenshots are mechanism-readability proofs.

3.2 Target-field construction

The target is constructed in a canonical wave frame. The canonical frame removes `position` and `steer`; the shape is computed there; then `steer` and `position` are applied as rigid transforms afterward. This prevents the old bug where moving the wave also changed its profile.

The source sequence in `buildNodes` is:

1. build rest grid p_{ij}^0 ;
2. compute core target positions from `targetFromDeformationField`;
3. pin perimeter nodes to rest;
4. apply span continuity smoothing for generated mode;
5. apply flat-contribution diffusion as a support apron, not as a flattening blend;

6. interpolate from rest to target with `morphGrowthAmount`;

7. build metrics, edges, quads, and dihedral pairs from the resulting grid.

The target function samples u along the wave field and v across span:

$$u = \frac{x - x_{\min}}{L}, \quad v = \frac{y + S/2}{S}.$$

The displacement is zero outside the support field and on the perimeter. The support field expands with smoothing and `flatContribution`, but the core wave must remain the same shape.

3.3 Reconstruction-grade mathematical model

Let the rest sheet be a rectangular lattice with R rows and C columns. In source, $R = 44$, $C = 112$, rest sheet length $L_s = 80$, rest sheet span $S_s = 80$, wave-field length $L = 43$, and wave-field span $S = 43$.

A node is

$$p_{ij}^0 = \begin{bmatrix} -L_s/2 + jL_s/(C-1) \\ -S_s/2 + iS_s/(R-1) \\ 0 \end{bmatrix}, \quad 0 \leq i < R, \quad 0 \leq j < C.$$

Boundary nodes are hard pinned:

$$p_{ij}^* = p_{ij}^0 \quad \text{if } i \in \{0, R-1\} \text{ or } j \in \{0, C-1\}.$$

For interior nodes, WaveVis maps the world point into a canonical wave frame by removing user placement:

$$\begin{aligned} \tilde{p} &= R_z(-\psi) \left(p^0 - a - \delta_x e_x \right) + a, \\ \psi &= \frac{\pi}{4} \text{clamp}(\text{steer}, -1, 1), \\ \delta_x &= 0.045L \text{clamp}(\text{position}, -1, 1). \end{aligned}$$

The canonical target is

$$\tilde{p}^* = \tilde{p} + D(\tilde{x}, \tilde{y}; \theta),$$

and the world target is restored by applying the inverse rigid placement. This separation is required because inverse-design controls should not silently change the target surface. It follows the same general separation used in graphics simulators between state variables, constraints, and display transforms (7,

9). 314

The active source contains two target/display paths that must not be confused: 315

1. **Model target path**, owned by `src/latticeGeometry.ts`. It generates node targets, metrics, and the mechanism-check state. 316
317
2. **Readable display path**, owned by `src/LatticeViewer3D.tsx`. It draws a reference-like gridded surface and mechanism layer for visual inspection. It is not allowed to replace the model target or hide invalid mechanism state. 318
319
320

Custom profile and readable-view split 321

When `profileMode='custom'`, the model source uses serialized control points in `src/latticeGeometry.ts`. The July 1 side-profile repair changed the model default to a mono-tone broad-rise profile rather than the older hooked point list: 322
323
324

(0, 0), (0.035, 0.006), (0.075, 0.026), (0.125, 0.08), (0.19, 0.18),
(0.27, 0.33), (0.36, 0.52), (0.46, 0.70), (0.56, 0.86), (0.65, 0.96),
(0.72, 1), (0.79, 0.98), (0.85, 0.90), (0.90, 0.75), (0.935, 0.55),
(0.955, 0.36), (1, 0).

Those are normalized (x, z) anchors for the model target, not final mechanism coordinates and not proof of reference match. They are sampled, scaled, localized across the span, morph-interpolated, and then checked against the mechanism constraints. 325
326
327

The readable display path in `src/LatticeViewer3D.tsx` now has a separate side-view trace. The 328

329 current side trace is a smoothed, normalized extraction from Figure 3:

(0, 0), (0.029, 0.046), (0.052, 0.091), (0.073, 0.136), (0.092, 0.181),
 (0.108, 0.227), (0.124, 0.274), (0.138, 0.321), (0.152, 0.368), (0.164, 0.416),
 (0.177, 0.465), (0.189, 0.515), (0.201, 0.565), (0.213, 0.616), (0.227, 0.666),
 (0.242, 0.715), (0.258, 0.763), (0.277, 0.808), (0.299, 0.85), (0.324, 0.889),
 (0.352, 0.924), (0.385, 0.954), (0.422, 0.978), (0.463, 0.995), (0.505, 1),
 (0.545, 0.991), (0.584, 0.968), (0.619, 0.936), (0.649, 0.899), (0.674, 0.86),
 (0.695, 0.817), (0.712, 0.773), (0.726, 0.726), (0.739, 0.678), (0.75, 0.627),
 (0.758, 0.574), (0.757, 0.52), (0.748, 0.468), (0.735, 0.431), (0.725, 0.424),
 (0.721, 0.452), (0.72, 0.5), (0.72, 0.554), (0.715, 0.609), (0.7, 0.659),
 (0.673, 0.7), (0.637, 0.727), (0.598, 0.74), (0.563, 0.733), (0.534, 0.709),
 (0.512, 0.671), (0.495, 0.623), (0.484, 0.57), (0.478, 0.515), (0.477, 0.461),
 (0.482, 0.409), (0.49, 0.358), (0.503, 0.31), (0.52, 0.265), (0.54, 0.223),
 (0.564, 0.184), (0.591, 0.149), (0.621, 0.117), (0.654, 0.089), (0.69, 0.065),
 (0.728, 0.044), (0.769, 0.028), (0.812, 0.015), (0.857, 0.007), (0.904, 0.003),
 (0.953, 0.003), (1, 0).

330 The side trace is allowed to backtrack enough to form a rounded curl, but it is not allowed to collapse
 331 into a needle. The checker computes a terminal backtrack ratio

$$\rho_{\text{back}} = \frac{\sum_{k=k_c+1}^{n-1} \max(0, x_{k-1} - x_k)}{\sum_{k=k_c+1}^{n-1} |x_k - x_{k-1}|},$$

332 where k_c is the crest index. The July 1 source gate rejects $\rho_{\text{back}} < 0.24$, rejects a low backtracked turn
 333 below $0.16H$, and uses a side-reference descent limit of 82° so the supplied curl can keep its tight inner
 334 sweep without allowing the old base cusp. The current side trace reports $\rho_{\text{back}} = 0.2658$, max terminal
 335 descent 81.0851° , max low-branch angle jump 7.6085° , and no readable side-profile blocker failures.
 336 The isometric, top, and front readable product views do not use that side trace as a swept body. They

use the localized three-dimensional/isometric trace:

337

(0, 0), (0.04, 0.008), (0.095, 0.033), (0.17, 0.105), (0.275, 0.305),
 (0.385, 0.58), (0.5, 0.82), (0.61, 0.965), (0.69, 1), (0.78, 0.965),
 (0.858, 0.86), (0.922, 0.704), (0.967, 0.528), (0.985, 0.384),
 (0.965, 0.3), (0.937, 0.228), (0.919, 0.16), (0.944, 0.112),
 (0.969, 0.061), (0.994, 0.012), (1, 0).

This split is deliberate. A profile broad enough for the human side-view read can become a top-view
 tunnel or wall if swept across the whole sheet. A valid future change must state which trace it changes,
 why, and which rendered view failed before the edit.

338

339

340

Generated lip path

341

The generated path uses a ramp/crest/lip decomposition:

342

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \leq u < u_c, \\ P_{\text{crest}}(u), & u_c \leq u < u_l, \\ P_{\text{lip}}(u), & u_l \leq u \leq 1. \end{cases}$$

In source, `moanaReferenceLipPoints` builds a normalized list of lip anchors controlled by `dip` and
`sharp`. The anchors are smoothed into cubic Bezier segments, then sampled by normalized arc length:

343

344

$$q(\alpha) = B_k(t) \quad \text{where} \quad \sum_{\ell < k} \int_0^1 \|B'_\ell(s)\| \, ds + \int_0^t \|B'_k(s)\| \, ds = \alpha \ell_{\text{total}}.$$

The source samples this curve numerically using 18 subdivisions per cubic segment. A future exact
 reconstruction should replace that sampling with an analytic or higher-resolution arc-length inversion
 only if the rendered mechanism improves and the validation gates still pass.

345

346

347

Span localization

The readable display path uses a lateral envelope

$$E(s) = \cos^{1.42} \left(\frac{\pi}{2} |s| \right), \quad s \in [-1, 1],$$

then uses powers of E near the terminal lip to localize the curl. The source intentionally makes the lip narrower than the broad ramp; otherwise a centerline curl becomes a swept tunnel. The July 1 repair keeps the lip round early, but releases it back to the flat body near the terminal endpoint:

$$r(t) = S(0.84, 0.98, t),$$

$$\tau(t) = \text{clamp}_{[0,1]} \left(t - 0.07 \sin [\pi S(0.68, 1, t)] [1 - S(0.46, 0.78, S(0.68, 1, t))] \right),$$

$$F_{\text{fold}}(s, t) = \text{lerp} \left(F_{\text{shoulder}}, E(s)^{1.78}, 0.42 S(0.70, 1, t) \right),$$

$$F_{\text{lip}}(s, t) = \text{lerp} \left(E(s)^{1.18}, E(s)^{2.15}, 0.82 S(0.70, 1, t) \right).$$

The endpoint release $r(t)$ also multiplies the curl interpolation and open-core lip blend, so the suspended lip does not drag a curl all the way to the flat terminal sheet. The repair also keeps a crest-cap exception around the rounded lip:

$$C(t) = S(0.54, 0.66, t) [1 - S(0.82, 0.94, t)],$$

$$H_{\text{cap}}(s) = (1 - S(0.20, 0.82, |s|))^{0.72},$$

and blends the height span toward H_{cap} with weight $0.97C(t)$. This is not a tunnel relaxation. It is the blocker against the opposite failure Alan identified: a pointy fin or spike in the front/cross-section view. In the model path, the same localization principle appears through `transverseSupportMask`, `lipAwareRimMask`, `terminalLipSpanMask`, `shapeMask`, and the post-core taper terms.

Bowl/dimple blocker

364

The source blocker `blockSurfaceBowlPockets` runs before node emission. For each interior node with active neighboring height, it replaces a local low point by the minimum of its four direct neighbors:

365

366

$$z_{ij}^{(k+1)} = \max \left(z_{ij}^{(k)}, \min(z_{i-1,j}^{(k)}, z_{i+1,j}^{(k)}, z_{i,j-1}^{(k)}, z_{i,j+1}^{(k)}) \right),$$

for up to six passes. This is a pre-render invariant sweep, not a visual cleanup. It catches bowl-like local minima, but it is not sufficient for the under-overhang support-neck failure; that failure also requires side/isometric throat sampling and human-facing render inspection.

367

368

369

Connected X-cell equations

370

For each node center c_n , the visible cell is one square body with four legs in directions $\{\text{NE}, \text{SW}, \text{NW}, \text{SE}\}$. For each opposite pair,

371

372

$$\|e_{\text{NE}} - c_n\| = \|e_{\text{SW}} - c_n\|, \quad e_{\text{NE}} + e_{\text{SW}} = 2c_n,$$

and similarly for NW/SE. The same connector point g_{ab} is shared by exactly two adjacent cells:

373

$$e_{a,+} = g_{ab} = e_{b,-}, \quad \#\{(n, d) : e_{n,d} = g_{ab}\} = 2.$$

The current source solves connector chains by alternating signs along each row/column family:

374

$$g_0 = f, \quad g_k = 2c_k - g_{k-1} \quad (k > 0),$$

then chooses f by least-squares midpoint residual over the adjacent pair family. This preserves exact cell-level opposite-pair closure and exposes center-corridor drift as a diagnostic rather than hiding it. The checker then groups connector positions by physical coordinate, not only by id, so two connector ids occupying one point cannot create a silent four-cell joint.

375

376

377

378

Acceptance objective

WaveVis is not currently optimizing a single scalar objective. The useful mental model is a constrained inverse-design problem:

$$\min_{\theta, \mathcal{G}} w_r E_{\text{ref}}(\Phi_\theta, \Phi_{\text{ref}}) + w_s E_{\text{smooth}} + w_m E_{\text{mechanism}} + w_v E_{\text{view}}$$

subject to boundary pinning, topology completeness, connector occupancy, no bowl/dimple pockets, and public route verification. At this stage the objective is enforced as explicit gates rather than a continuous solver: reference match is judged on the rendered object; mechanism feasibility is checked in source; stale paper/public assets are treated as failures.

For a future continuous solver, the scalar terms should be reconstructed from the current gates rather than invented from scratch. A practical decomposition is:

$$\begin{aligned} E_{\text{ref}} &= \lambda_s D_{\text{side}}(S_{\text{side}}, R_{\text{side}}) + \lambda_t D_{\text{top}}(S_{\text{top}}, R_{\text{top}}) + \lambda_f D_{\text{front}}(S_{\text{front}}, R_{\text{front}}) + \lambda_o E_{\text{open}}, \\ E_{\text{smooth}} &= \sum_{(i,j)} \|\Delta_x^2 p_{ij}\|^2 + \|\Delta_y^2 p_{ij}\|^2, \\ E_{\text{mechanism}} &= \sum_n \left(\epsilon_{\text{oppLen}}(n)^2 + \epsilon_{\text{oppCol}}(n)^2 + \epsilon_{\text{jointUse}}(n)^2 + \epsilon_{\text{anchor}}(n)^2 \right), \\ E_{\text{view}} &= \epsilon_{\text{sideWall}}^2 + \epsilon_{\text{skinnySail}}^2 + \epsilon_{\text{frontPoint}}^2 + \epsilon_{\text{frontWall}}^2 + \epsilon_{\text{pointyProfile}}^2 + \epsilon_{\text{hiddenMech}}^2. \end{aligned}$$

Here $R_{\text{side}}, R_{\text{top}}, R_{\text{front}}$ are view-level targets extracted from the accepted reference family. R_{side} may use the July 1 supplied side line; $R_{\text{top}}, R_{\text{front}}$, open-throat, perimeter, and mechanism terms must still come from the gridded and three-dimensional references. D_{side} may include a Frechet-like curve-distance term after a side contour is extracted from the full sheet, but that term is invalid if the top, front, open-throat, or mechanism-visibility terms fail. E_{open} samples the underside throat below the lip for dimples, bites, bowls, and support-necks, and D_{top} rejects broad terminal blobs, triangles, and V fans in plan view. $E_{\text{frontPoint}}$ rejects a rounded-wave front view that collapses into a vertical fin; $E_{\text{frontWall}}$ rejects the opposite full-width wall; $\epsilon_{\text{pointyProfile}}$ rejects a sharp or needle-like side silhouette even when the outer curl radius appears continuous. This follows the general inverse-design pattern of optimizing a physically meaningful state subject to explicit manufacturability and simulation constraints (10, 22–24), but WaveVis should keep the gate form until the rendered artifact is stable enough that a scalar objective would not hide a visible failure.

Candidate iteration record

Every new geometry candidate must be traceable. The minimum record is:

$$\mathcal{R}_k = \{\text{candidate id, source diff, intended defect, metrics, render captures, accept/reject reason}\}.$$

The accept/reject reason must name the rendered shape, not only the code change. A valid note says, for example, “rejected: side open throat improved but top became a broad terminal blob.” An invalid note says only “geometry check passed.” Candidate848 through Candidate875 are examples of why this record matters: some local gates passed, but browser renders still showed support ridges, broad top blobs, front-wall reads, pointy fin profiles, squared terminal feet, or support-cone reads. Candidate876 is the current committed baseline only; its old build pass and geometry-invariant pass do not override the later rendered pointy-profile complaint. Candidates 898 through 903 extend the rejection ledger: old smooth trace kept a spiral/pointy side and pinched top; tail release became a blunt cylinder; smooth product profile became a loop/half-pipe; no-loop cap became a mound with no curl; Moana lip anchors made a steep nose without the needed overhang; analytic rounded curl became a broad wall/blob. The current July 1 repair must be recorded as a separate source state: it passes the inverse-sheet source contract and removes the pointy side profile locally, but its rendered full-sheet reference match remains open unless the current public routes are inspected after the bundle update.

3.4 Rigid controls

position and steer are post-shape placement controls:

$$\Delta x_{\text{position}} = 0.045 L \text{ clamp}(\text{position}, -1, 1),$$

$$\psi_{\text{steer}} = 45^\circ \text{ clamp}(\text{steer}, -1, 1).$$

They must not modify the curated wave profile, width, curl, strain, or topology. Validation aligns geometries back to the canonical frame and checks that residual shape differences are near zero for position and bounded for steer.

3.5 Morph control

morph is not a shape recipe. It interpolates from rest to the completed target:

$$p_{ij}(m) = (1 - g(m))p_{ij}^0 + g(m)p_{ij}^*.$$

The current growth function combines a sine ease with a faster visible growth curve so the wave grows naturally without a dead early range. Morph must preserve topology, connector closure, and the visible wave identity throughout the range. If morph = 0.5 produces zig-zags, overlaps, missing cells, or a side-wall-covered curl, that is a geometry failure.

3.6 Flat contribution and ground transition

flatContribution does not mean flatten the wave. It means share deformation with the surrounding sheet through a support apron. In accepted behavior:

- the main wave height and overhang reach stay within a small residual;
- the centerline profile remains essentially unchanged;
- surrounding flat areas participate more gradually;
- strain concentration is reduced or spread rather than hidden.

The old behavior target = lerp(deformed, rest, flatContribution) is explicitly rejected because it turns flatContribution into a flattening slider.

smoothing, shown historically as ground transition, broadens the support field and root ramp. It should recruit surrounding flat sheet into the slope while keeping the perimeter pinned and preserving the terminal lip.

3.7 Lip dip and terminal curl

lipDip maps to overhangAngleDeg. Neutral is 90°. High curl is 120°. The accepted behavior is not endpoint lowering. The wave should rise, crest, reach forward, curl downward, then return smoothly to the flat sheet without closing into a bowl. The current validation checks:

- the selected lip nose is a mid-height visible point below the last peak, not the near-floor return;
- the terminal slope is negative;

- the tip is forward of the crest and tucked under the shoulder by a bounded amount; 445
- the return advances toward the flat front instead of folding backward; 446
- the terminal cavity blocker, backfold blocker, and visible-dimple blocker remain true. 447

These checks are guards, not substitutes for the human-facing screenshot. A visible dimple, under- 448
overhang pocket, side wall, or missing curl still means open work. 449

3.8 Rendering contract 450

LatticeViewer3D.tsx renders the readable surface and a connected X-cell mechanism from the 451
actual lattice graph. In the current contract: 452

- `view=side` sets a side camera over the full three-dimensional linkage array; 453
- `view=front` sets a front camera over a crest-section window sampled from the same readable 454
three-dimensional field, so the view inspects the rounded cross-section instead of projecting the 455
whole side trace into a fin; 456
- `view=isometric` shows the full rectangular sheet in perspective while still making the curl and 457
lip readable; 458
- the default product view uses a smooth gridded surface skin plus one-center four-leg X-cell leg 459
lines, black square cell bodies, and spherical connector nodes; 460
- Side, Front, and 3D use view-specific mesh outlines instead of the previous SVG overlay, centerline 461
comparison, or red terminal trace; 462
- each rendered X cell is one black square cell body with four visible center-to-connector legs; 463
- each connector is a spherical two-use physical node between adjacent cells; 464
- each connector has exactly two endpoint uses: one leg from each of those two centers; 465
- crossing diagonal families must stay visually and physically separate instead of collapsing into a 466
four-cell joint; 467
- connector corridor drift is reported as a bounded diagnostic when exact cell-level opposite-pair 468
closure moves a connector away from the old midpoint corridor; 469

- cells and X legs are display toggles, not repair paths, and their render scope remains covered by the geometry check.

If the renderer needs to hide rows to make the shape readable, the generator is still wrong.

4 Target outcome

4.1 Human-facing shape

The accepted target is a “flat sheet plus localized plunging wave”:

- the outer perimeter of the rectangular sheet remains flat and fixed;
- the wave emerges through a broad, smooth ramp rather than a vertical wall;
- the crest is rounded, with no top dent;
- the lip projects forward and curls downward;
- the open underside is visible from side view, not covered by a side wall;
- the shape fades back to flat in every direction;
- the array remains a full linkage field with nodes and legs, not a hidden or clipped shell.

The practical acceptance boundary is a projected full-sheet condition, not a standalone drawn profile.

Let the rest sheet be $\Omega = [-1, 1] \times [-1, 1]$ with boundary $\partial\Omega$, sampled by linkage nodes $r_{ij} = (x_i, y_j, 0)$, and let the generated mechanism state be $p_{ij} = \Phi_\theta(r_{ij})$. A future candidate is eligible for visual review only if

$$\max_{r_{ij} \in \partial\Omega} \|p_{ij} - r_{ij}\| \leq \varepsilon_\partial, \quad \mathcal{V}_{\text{mech}}(p, \mathcal{G}) = 1,$$

where $\mathcal{V}_{\text{mech}}$ means full visible cell bodies, four straight legs per cell, two-use connector joints, and no hidden filler geometry. The side, top, front, and isometric views are then projections of the same state:

$$S_{\text{side}} = \Pi_{xz}\{p_{ij}\}, \quad S_{\text{top}} = \Pi_{xy}\{p_{ij}\}, \quad S_{\text{front}} = \Pi_{yz}\{p_{ij}\}.$$

The candidate must satisfy all three projected readings at once: broad rounded side curl, rounded localized top footprint, and rounded front/cross-section body. A side-line surrogate is rejected even if it looks clean in isolation, because it cannot encode span localization, open throat clearance, perimeter

flatness, or mechanism visibility.

4.2 Hard constraints

The WaveVis generator and renderer must preserve the following hard constraints:

1. **Flat perimeter.** Boundary nodes must remain on the rest sheet. No side, front, or rear perimeter should be pulled into the wave.
2. **Full array visibility.** The rendered artifact must show the actual linkage grid. Hiding or ghosting broken cells does not count as a fix.
3. **Connector closure.** A cell connector is an actual shared graph location. Lines should meet at nodes; apparent detached legs are a failure.
4. **No erratic legs.** Long arms, spikes, zig-zags, random backtracking, and visible overlaps indicate a broken geometry or render path.
5. **Smooth strain sharing.** Adjacent rows and columns should change gradually enough that the mesh reads as a continuous mechanism field.
6. **Localized wave.** The curl must be strongest near the center and fade laterally. A constant side-to-side tunnel is not the target.
7. **Full-view side semantics.** Side view shows the full three-dimensional render from the side. It must not be replaced by a profile-only debug view.
8. **No filler geometry.** Cosmetic walls, caps, planks, bridges, hidden couplers, or masks must not be used to conceal a bad mechanism.

5 Coordinate model

The simulator starts from a rectangular rest lattice. A node with row i and column j has rest position

$$p_{ij}^0 = (x_j, y_i, 0).$$

The target is a displacement field

$$p_{ij}^* = p_{ij}^0 + m D(x_j, y_i; \theta),$$

514 where $m \in [0, 1]$ is the morph amount and θ represents all shape parameters except rigid placement.
 515 The morph parameter only interpolates between rest and the finalized target. It must not change the
 516 target shape definition, remesh the lattice, or introduce a different topology.

517 The longitudinal coordinate $u \in [0, 1]$ measures progress through the wave region from rear/root to
 518 front/lip. The span coordinate $v \in [-1, 1]$ measures lateral distance from the centerline. A good
 519 WaveVis target uses both:

$$D(x, y) = A(u, v) P(u) + B(u, v) Q(u),$$

520 where $P(u)$ is the side-profile contribution, $Q(u)$ can include forward curl or secondary displacement
 521 terms, and A, B are smooth two-dimensional envelopes that go to zero at the sheet perimeter.

522 The key architectural rule is that $A(u, v)$ is not constant in v . A constant span envelope turns any
 523 attractive profile into a tunnel. A localized plunging wave requires $A(u, 0)$ large near the centerline
 524 and $A(u, \pm 1) \approx 0$ near the sides, with a monotone or smoothly unimodal falloff rather than a hard wall.

525 6 Generation pipeline

526 The target-building pipeline is:

- 527 1. sanitize rows, columns, visible sliders, hidden legacy shape parameters, display flags, and URL
 528 parameters;
- 529 2. build the full rectangular node grid;
- 530 3. compute the canonical rounded breaking-wave displacement from the supplied reference target;
- 531 4. apply the span envelope that localizes the wave in y ;
- 532 5. preserve the boundary by forcing perimeter displacement to zero;
- 533 6. apply morph interpolation from rest to target;
- 534 7. apply rigid steer and position transforms after the shape is finalized;
- 535 8. build full rectangular edge and quad topology;
- 536 9. compute strain, bend, shear, dihedral, and display metrics;

10. render the full array, side view, 3D view, and top view without substituting fake surfaces. 537

The distinction between the supplied reference target and rigid placement parameters is essential. The 538
visible app no longer exposes manual xz or xy profile editors. The source target owns the rounded 539
breaking-wave profile. The user-facing sliders are intentionally narrow: scale uniformly resizes that 540
target, morph blends from rest to the completed target, steer rotates the finished target around the 541
anchor, and position translates the finished target horizontally. Neither steer nor position may feed 542
back into profile curvature, curl, width, strain generation, or cell topology. 543

Stage	Contract	Failure caught
Target profile	Defines the desired side profile shape.	Blob, neck/head, hill without lip, top dent.
Span envelope	Localizes the wave in the sheet.	Swept barrel, side wall, U-shaped tube blocker.
Boundary clamp	Keeps the perimeter flat.	Pulled sheet edges, non-flat perimeter.
Morph	Interpolates rest to target only.	Half-morph mangling, topology change while sliding.
Topology	Keeps full node/edge/quad graph.	Missing grid, hidden cells, detached legs.
Renderer	Shows the actual mechanism.	Fake shell, hidden wall, side view as profile-only debug output.
Paper evidence	Includes only target references, accepted mechanism proofs, accepted current renders, or clearly labeled rejected diagnostics.	Stale candidate screenshots, misleading overlays, and internal trace diagrams appearing as authoritative figures.

Table 3. Pipeline stages and the failure modes each stage must prevent.

7 Supplied rounded target 544

The current visible app does not ask the user to draw a profile. That path created too much ambiguity 545
and repeatedly made the simulator look like a swept tunnel or hand-edited cavity rather than a single 546
coherent wave. The user-facing target is now driven by the supplied rounded breaking-wave lip/body 547
profile: a localized ocean deformation with a broad rear ramp, rounded crest, forward-and-down lip, 548
open underside, smooth lateral fade, and flat rectangular perimeter. 549

The curated target is a reference field, not a set of cell locations. The final cell grid is determined by 550
row and column counts, then sampled against that target. A clean state must keep the full linkage array 551
visible while making the centerline and lateral profile read as one plunging wave. The target must 552

never stitch the lip directly to the ground or draw a hard polygon boundary around a former editor curve, because that creates the wall, bowl, cavity, and silhouette-only failures this record explicitly rejects. The curl is also resolution-sensitive: the default reference-wave state keeps 112 columns, while reduced column counts are reserved for low-density mechanism inspection and must not be mistaken for a completed reference-shape view.

8 Slider semantics

Control	Intended effect	Must not do
morph	Blend rest sheet into the completed target.	Remesh, flip cells, create zig-zags, change profile identity.
scale	Uniformly enlarge or reduce the supplied rounded target.	Change steer, position, topology, or the target's wave identity.
steer	Rigid yaw rotation of the finished wave.	Bend, stretch, or reshape the wave.
position	Rigid horizontal translation of the finished wave.	Change profile, strain field, or curl.
display toggles	Surface, flat grid, cells, and X legs are display layers only.	Hide broken geometry or substitute a fake repair path.

Table 4. Visible control contract after removing manual profile editors and hidden shape sliders. The supplied rounded target is the default shape; visible controls must preserve that shape unless their narrow role explicitly changes scale, morph, steer, position, or display layers.

9 Breaking-wave geometry

The desired default profile is closer to a localized plunging wave than to a cone, cylinder, or arch. The useful approximation is a tapered, asymmetric surface whose highest crest sits behind the curl, whose outer radius is larger than the inner radius, and whose underside remains open. Along the centerline, a breaking wave profile can be represented as three joined curve families:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \leq u < u_c, \\ P_{\text{crest}}(u), & u_c \leq u < u_l, \\ P_{\text{lip}}(u), & u_l \leq u \leq 1. \end{cases}$$

The ramp rises from the sheet. The crest rounds over. The lip turns forward and downward. The derivative at the terminal lip should point downward when $\text{lipDip} > 90^\circ$:

$$\left. \frac{dz}{dx} \right|_{\text{tip}} < 0.$$

The final point is allowed to be lower than the farthest horizontal reach. In a real breaking wave, the maximum reach can occur at the shoulder while the lip tip turns down from that shoulder. The downturn must remain open and forward-readable from the side; it must not fold into a closed pucker, dimple, bowl, or side-wall pocket. Treating the farthest-right point as the tip is the mistake that produces a straight falling chute, while overcorrecting into an under-tucked return is the mistake that produces the rejected cavity. The underside return must also advance back into the flat sheet after the lowest lip sample; a backward floor point is disallowed because it reads as a small lower-lip dimple even when it does not trip a larger bowl metric. Candidate785 was the previous recovery floor for this idea, not the accepted answer; the current acceptance test is still the rendered full mechanism with an open throat and no support-neck pocket.

The three-dimensional field must multiply this centerline motion by a span falloff:

$$E(v; u) = \exp \left[- \left(\frac{|v|}{w(u)} \right)^\alpha \right],$$

with $w(u)$ varying through the wave. The curl should have a smaller active width than the broad back ramp, so the lip is localized and the open underside remains visible from the side.

10 Mechanism topology

WaveVis uses a rectangular graph of nodes, horizontal profile edges, vertical span edges, and quads as the source lattice. The visible mechanism layer then derives a connected X cell at each lattice node. This is a kinematic-design problem, not a drawing style: linkage synthesis and symbolic kinematics treat joints, bars, and closure equations as the object being designed (15–17). The WaveVis renderer must therefore show the constraint truth rather than draw a smoother surrogate.

Each cell renders as a black square body with four visible legs that leave the same center. Opposite leg pairs are equal length and collinear through that center, so the center bisects both bars. Each leg terminates at a spherical connector node that is shared by exactly two adjacent cells. Crossing

diagonal families are separated into distinct physical joints so a crossing point cannot become a four-cell connector. The graph is not allowed to become isolated crossed marks, two nearby two-leg pseudo-cells, or a set of isolated rows.

The mechanism evidence from the earlier proof remains relevant as a warning against fake filler surfaces and impossible all-four-equal closure. The current mechanism rule is cell-level, not candidate-level: each cell has one square center and four straight legs, opposite pairs are collinear and equal length through that center, and each connector is a spherical physical joint used by exactly two adjacent cells. The old center-pair corridor is a drift diagnostic, not the hard pass condition (28).

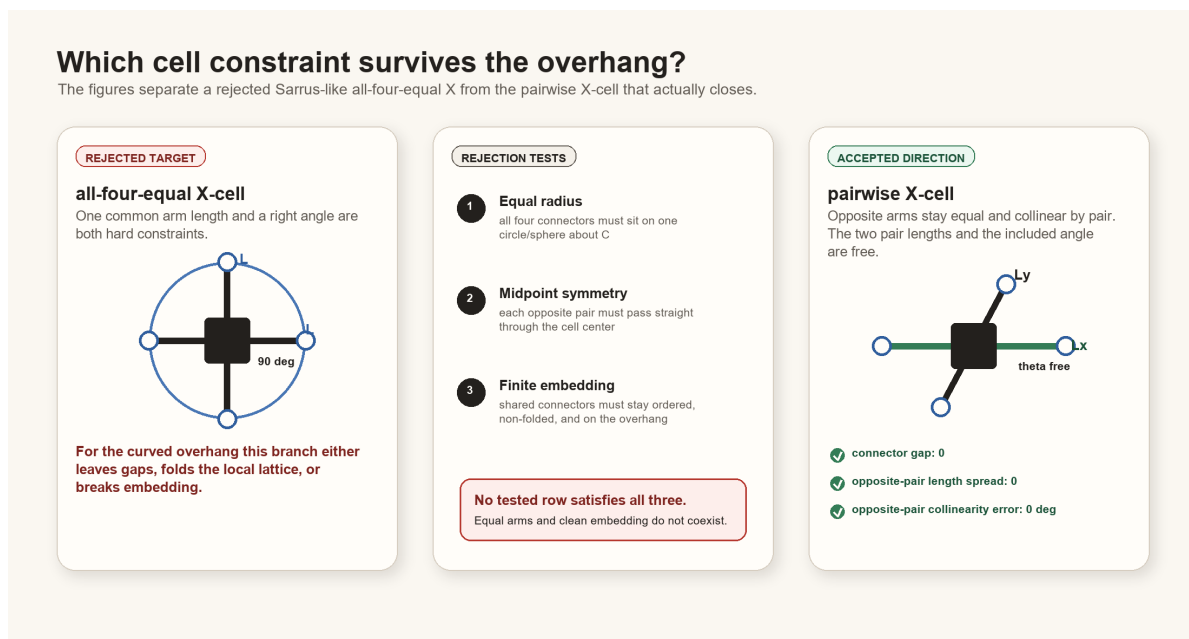


Figure 7. Mechanism contract inherited from the constraint proof and updated in the current renderer. WaveVis should preserve exact shared contact, one-center cells, and two-cell connector joints on adjacent-center spans rather than hiding infeasible geometry behind filler surfaces or a connector-ID count.

11 Rendering modes

WaveVis has three inverse-sheet product views:

- **3D/isometric view** shows the full rectangular array in perspective.
- **Side view** is a camera view of the full three-dimensional render from the side and must show the curl.
- **Top view** shows the full square plan sheet with localized interior deformation and catches terminal caps, triangular fans, wedges, or cropped-patch regressions.

The view contract matters because a side view can hide the curl if lateral geometry forms a wall over the open underside. In that case the correct fix is to reshape the three-dimensional surface so the lateral envelope follows the wave contour and fades out, not to switch the side view into a profile-only debug output or hide the wall.

11.1 Magnetic valve sandbox

The July 2026 advisor-call transcript added a separate design need: a fast sandbox for a small magnet-pneumatic valve before committing to heavier COMSOL-style work. This is not a replacement for finite-element magnetics or measured flow validation. It is a build-selection tool for the first physical decisions: tube inner diameter and length, steel-ball diameter and friction, left/right barbed fitting orifice size, NdFeB and Alnico placement, steel pole-plate placement, coil-wrapped regions, pulse polarity, winding reversal, pressure level, and flow direction.

The tab named `Magnetic Valve` renders the whole valve assembly as the primary object. The default preset is a short tube with 1/16-inch-class barbed seats, a ferrous ball, NdFeB and Alnico magnets, steel pole plates, and a coil already wrapped around the Alnico region. The compact control strip exposes only the live operations Alan needs while designing: flow on/off, PSI, flow direction, coil wrapping, pulse +, pulse -, and reverse winding. Selecting the tube, ball, seat, magnet, plate, or coil opens only that object's controls; dragging across the magnetic stack creates a coil wrap.

The client-side model keeps the physics deliberately approximate. NdFeB components have fixed magnetization. Alnico components store a programmed direction and strength from the selected coil's current, pulse time, pulse sign, and winding direction. Steel plates apply a bounded local flux-concentration multiplier. The ball force is estimated from the one-dimensional magnetic gradient along the tube. Pneumatic pressure applies force through the active seat orifice area, while tube/orifice resistance estimates remaining flow. Reversing flow moves the active blocking seat; when the ball seats at that active orifice, downstream flow arrows fade, upstream pressure is shown, and the result strip reports flow cut off, leaking, or flow open. A magnetic pull that unseats the ball immediately returns the visual state to open flow.

12 Verification gates

The generator should be checked with both numeric and visual gates. Numeric checks are not sufficient, but they catch repeated hard failures before visual review.

Gate	Measurement	Acceptance intent
Perimeter flatness	$\max_{\partial\Omega} \ p^* - p^0\ $.	Boundary displacement is zero or visually negligible.
Topology completeness	Expected node, edge, and quad counts.	Full rectangular sheet graph remains visible.
Connector closure	Edge endpoints meet their node coordinates.	No detached legs or fake bridges.
Lip curl	Crest-to-lip height gap, terminal slope, run gate, and cavity gate.	The lip curls down into a smooth tapered branch rather than ending as a hill, near-vertical chute, hooked tip, bowl, dimple, or cavity.
Span localization	Row-wise height profile across y .	Center strong, sides faded; no swept tunnel.
Morph continuity	Residual between adjacent morph samples.	No sudden jumps or topology flips.
Slider isolation	Aligned geometries across position/steer/width changes.	Rigid controls stay rigid; width stays span-only.
Visual side check	Browser side view screenshot or direct render inspection.	Curl visible in the full side view.
Defect repair loop	Confirmed visible defects re-enter source repair.	Acknowledging dimples, pockets, walls, or missing curl is not a stopping state.
Terminal side-profile gate	Literal readable profile, sampled model side trace, terminal backtrack ratio, and side camera fit.	Before visual QA, reject any lower-branch re-rise, under-lip valley rebound, pointy-notch under-backtrack, excessive lower-branch backtrack, abrupt terminal angle jump, steep visible terminal descent, or cropped mobile side view.
Startup URL parser	Covers all public and legacy slider aliases.	A shared verification URL cannot silently load a different height, overhang, lip dip, width, or smoothing state.
Paper figure hygiene	Every figure is a target reference, accepted mechanism proof, current accepted render, or labeled rejected diagnostic.	Internal overlays, stale screenshots, and profile-only traces cannot become authoritative paper evidence.

Table 5. Verification gates. A build passing TypeScript is not enough; WaveVis must pass the rendered outcome Alan actually uses.

12.1 Current regression command set

The current inverse-sheet source-level gate is:

```
npm run check:geometry
```

which expands to:

```
node scripts/check-geometry-invariants.cjs
```

```
node scripts/check-inverse-sheet.cjs
```

```
node scripts/readable-wave-metrics.cjs
```

The inverse-sheet checker compiles the TypeScript geometry into a temporary CommonJS module and then evaluates the default state, low/high lip dip, morph, position, steer, flat contribution, smoothing, width, lateral taper, bowl-pocket, side-render, and startup-URL cases. The readable-wave metrics gate preserves the current reference-shape blockers. The geometry-invariant checker covers mechanism-level consistency.

The magnetic-valve sandbox has its own direct regression gate:

```
node scripts/check-magnetic-valve.cjs
```

That checker compiles the lightweight valve model and checks open flow, active-seat cutoff, reversed-flow seat selection, pressure scaling, tube/orifice resistance, magnet-placement bias, and Alnico pulse/winding direction.

The current acceptance evidence expected from `scripts/check-inverse-sheet.cjs` includes:

- startup display defaults to `showSurface=true`, `showEdges=true`, `showNodes=true`, and `showHeatmap=false`;
- rendered black-square cell count equals the full rectangular grid count 4928;
- rendered center-to-connector X leg segment count equals the expected connector-use count 19400;
- every interior X cell has exactly four legs from one black square body;
- rendered spherical connector-node count equals the expected adjacent connector count 9700;
- each physical connector has exactly two endpoint uses;

- 656 • maximum X connector endpoint gap is zero;
- 657 • maximum opposite-pair length spread is zero;
- 658 • maximum opposite-pair collinearity error is zero degrees;
- 659 • maximum opposite-center residual is zero;
- 660 • connector corridor drift is bounded as a diagnostic rather than used as the pass condition;
- 661 • `sourceUsesCollinearEqualPairConnectors=true`;
- 662 • connected X-cell bodies stay close to the sampled wave surface rather than becoming a detached
663 fake mechanism;
- 664 • no bowl-pocket count is reported for default, compact-user, and breaking-wide presets;
- 665 • the literal readable side trace, side/isometric readable profile, startup default, user morph half-lip
666 case, and slider sweeps all pass the terminal and readable side-profile cavity blockers before browser
667 screenshots are treated as useful evidence; the July 1 side trace reports $\rho_{\text{back}} = 0.2658$ against the
668 pointy-notch rejection floor 0.24;
- 669 • `blockSurfaceBowlPockets` runs on the target grid before morphing and node emission, and the
670 named 79-case slider sweep reports empty `badCases` arrays for the side-cavity and surface-bowl
671 gates;
- 672 • side-overhang aspect stays in the downturned-lip band after the supplied-line update: startup default
673 aspect between 1.35 and 3.05, compact user morph case between 1.45 and 3.15, and both with
674 reach-to-height between 0.72 and 1.35;
- 675 • broad open-curl lip stats keep open-throat, no-dimple, no-under-overhang-bite, no-backfold, and
676 return-to-flat checks true;
- 677 • those lip stats allow a wide rounded cap and tucked nose instead of forcing the old vertical-chute
678 proxy;
- 679 • slider robustness exits cleanly for the named acceptance sweeps, while printed diagnostic subcases
680 can still expose unsupported-resolution or terminal-angle failures that must not be described as
681 accepted visual states.

682 These checks are designed around Alan's repeated failures. If a future change removes one of these
cases because it is inconvenient, that is a regression unless the PDF and source explain an intentional
replacement gate that still catches the same failure. 683
684

12.2 Rendered verification 685

Numeric checks must be followed by rendered verification. The minimum visual pass is: 686

1. render the exact user URL/preset, not a nearby default; 687
2. check Side: full side camera, visible broad curl, no pointy side profile, no cropped mobile curl,
no wall covering the open underside, and no dimple or dark pocket under the overhang; 688
689
3. check 3D: full-sheet perspective view, visible curl/overhang, localized center deformation, no
swept tunnel, no hidden rows, and no underside bite hidden by the camera angle; 690
691
4. check Top: the sheet should remain square while the interior footprint localizes toward the
curl; it must not become a constant extruded strip, broad terminal cap, clipped patch, or large
triangular fan; 692
693
694
5. inspect cells/connectors at the curl for zig-zags, random long arms, detached legs, overlaps, or
missing four-leg interior connectivity; 695
696
6. check lower lipDip values as well as 120°, because old low-lip cases became unstable even
when max lip looked acceptable; 697
698
7. check morph=0, 0.5, and 1 because half-morph repeatedly exposed mangled intermediate states. 699

Screenshots are evidence only when they show the actual rendered canvas and the URL or visible
controls identify the tested state. A screenshot of a nice profile line is not evidence that the full linkage
array is correct. 700
701
702

13 Build, deploy, and publication runbook 703

13.1 Local development

704

Use the WaveVis app root:

705

```
_apps/wavevis.aolabs.io/.
```

706 Typical local commands are:

- 707 • `npm run dev` to run the Vite development server;
- 708 • `npm run check:geometry` to run geometry and mechanism checks;
- 709 • `npm run build` to build the static app;
- 710 • `npm run deploy` to publish the app's own GitHub Pages branch.

711 Do not manually edit `dist`. Build it from source.

712 13.2 Public fallback deployment

713 The route Alan can reliably use is `https://aolabs.io/wavevis/`. That route is served from the
714 AO Labs hub/fallback repository, not directly from the WaveVis app repository. Therefore a public
715 WaveVis update requires two publication steps when the fallback route is the user-facing route:

- 716 1. build and publish the WaveVis app;
- 717 2. copy the built `dist` contents into `_apps/aolabs-site/wavevis/`;
- 718 3. commit and push the AO Labs site mirror;
- 719 4. verify that `https://aolabs.io/wavevis/` serves the new bundle hash and the rendered
720 artifact.

721 This is not optional. A source commit or WaveVis `gh-pages` deploy can still leave Alan looking at a
722 stale `aolabs.io/wavevis/` bundle.

13.3 Custom-domain boundary

`https://wavevis.aolabs.io/` is the preferred branded route, but a hostname/certificate failure means it is not the verified artifact. The current Progress source separates `wavevis_home` from `wavevis_custom_domain`. Keep them separate. Do not claim the custom subdomain is fixed until a fresh request verifies the certificate and body from that hostname.

13.4 Progress logging

After meaningful WaveVis work, write a Progress event before final response when practical. The event must include:

- `source_ids`: usually `wavevis_home` and `wavevis_custom_domain`;
- Alan's complaint/request;
- the issue being solved;
- the Codex change;
- the observed public source change;
- provenance: thread, commits, checks, screenshots, and deployment basis;
- uncertainty: exact match not proven, custom domain stale, or any unverified surface.

Progress does not automatically ingest every active Codex chat. If a future WaveVis chat uses this PDF and changes the app, it must still write the event itself.

14 Current verified and open state

This section is intentionally blunt so a future chat does not have to infer whether the system is finished.

Verified current facts on July 1, 2026:

- The reliable public route is `https://aolabs.io/wavevis/`; the preferred HTTPS subdomain remains a separate certificate/live-route check.
- The Inverse Sheet tab is the first/default option; Double-Layer Sarrus is secondary.
- The public PDF route is `/wavevis/proofs/wavevis-system-architecture.pdf`.

- Side, Front, Top, and 3D are product view modes. Side is a full three-dimensional sheet view, not a profile-only debug trace.

- The current source tree contains the one-center four-leg X mechanism renderer, black square cell-body renderer, spherical connector-node renderer, and the geometry checkers that measure those invariants. The July 1 checker run reports the connected-X source contract as true:

```
xRenderDirectLines.sourceUsesConnectedXMechanism=true
```

- The July 1 side-profile repair passes local `npm run check:geometry`. The same run reports:

```
readableSurfaceRenderContract.ok=true
readableSideProfileCavityBlock.ok=true
readableTunnelSweepBlock.ok=true
```

and no inverse-sheet checker failures.

- The July 1 side-profile repair passes local `npm run build`; the built bundle is in `dist/`.

- Browser captures for the current local repair exist in:

```
_verification/
browser-20260701-
browser-20260701-side-reference-smoothed/
browser-20260701-side-long-tail/
```

The first folder contains the widened side desktop/mobile fit evidence; the second contains side, front, top, and isometric evidence before the final side framing change.

- The wrong internal reference-overlay figure and irrelevant stills have been removed from the paper figure sequence. The July 1 supplied side-view line is preserved as a side-only reference figure.

Open state that blocks completion:

- Candidate876 was previously source/build/PDF/live-route checked as a baseline, but it is superseded by the July 1 side-profile repair after the pointy-profile complaint.
- Candidate785, Candidate841 through Candidate875, and Candidate898 through Candidate903 are not final accepted states. They remain rejected or superseded diagnostics.
- The current local side profile is no longer the earlier pointy spear/notch, and the mobile side view is no longer cropped. Exact reference-family match is still not proven because the rendered full

mechanism must be judged across side, isometric, top, and front against the supplied gridded references.

- Exact matching to the June 24 gridded reference, June 28 stage/cross-section reference, July 1 side-view reference, or retained ocean-curl photographs is not proven. The July 1 side line constrains the side silhouette only; it is not a standalone acceptance target.

- This is a simulator and architecture record. It is not evidence that a physical lattice prototype has been fabricated or experimentally validated.

- If the live app, source, screenshots, Progress, or this PDF disagree, the current rendered artifact and source tree must be inspected and reconciled before any completion claim.

15 Failure modes

The most important failure classes are now explicit:

1. **Swept barrel.** One side profile is dragged sideways at constant span. The result reads as a tunnel, roof, half-pipe, or bridge.
2. **Side-wall cover.** The lip may exist in a center profile, but full side view shows a wall covering the open underside.
3. **Straight chute.** The crest drops nearly vertically to a flat shelf instead of curling forward and under.
4. **Blob or lump.** Excess smoothing or broad support creates a mound with no wave identity.
5. **Top dent.** The outer crest has a kink or concave dent where a smooth radius is required.
6. **Dimple, cavity, under-lip bite, or bowl.** The surface creates a pocket, pucker, enclosed hollow, under-overhang throat notch, or bowl-like depression rather than a plunging sheet.
7. **Dangling terminal connection.** The side view avoids the old dimple metric but leaves a visible lower terminal tongue or foot under the lip. This still fails because the reference curl has a smooth open throat and a flat sheet return, not a small support hook that visually wires the lip to the ground.

8. **Stale URL state.** A browser URL contains height, overhang, width, lipDip, or smoothing values, but startup ignores them and renders a different state.
9. **Zig-zag linkage.** Neighboring legs alternate direction or length erratically.
10. **Hidden mechanism.** Broken topology is ghosted, clipped, or replaced by a cosmetic surface.
11. **Pointy surrogate target.** A clean side line or internal contour is treated as the whole target even though it produces a pointy profile or does not define the gridded sheet, top footprint, front body, open throat, or visible mechanism.
12. **Wrong paper evidence.** A stale overlay, profile trace, internal debug diagram, or rejected candidate screenshot appears as if it is a correct reference or accepted result.
13. **Passive defect acknowledgement.** A visible problem is named in chat or recorded in this paper, but the generator and rendered artifact are not repaired and rechecked.

The solution direction is to simplify the target field while strengthening the gates: localized smooth wave first, full array second, mechanism-clean render third. Complexity should be added only if it improves one of those outcomes without breaking the others.

16 Materials and Methods

The implementation source is the WaveVis React and TypeScript app (29). The main inverse-sheet geometry source is `src/latticeGeometry.ts`. The primary inverse-sheet render source is `src/LatticeViewer3D.tsx`. Inverse-sheet controls are declared in `src/TargetShapeControls.tsx`. The magnetic-valve sandbox is declared in `src/MagneticValveTab.tsx` and its approximate coupled magnet-pneumatic model is declared in `src/magneticValveModel.ts`. Regression checks are run through `scripts/check-inverse-sheet.cjs`, `scripts/readable-wave-metrics.cjs`, and `scripts/check-magnetic-valve.cjs`.

The model is deliberately written as an inspectable engineering simulator rather than a black-box optimizer. The validation style borrows from topology-optimization practice, where a compact code can still be valuable when its filters, constraints, and objective terms are explicit (20, 30); from linkage and compliant-mechanism design, where target motion is inseparable from topology, joint placement, actuator choice, and manufacturable compliance (12–14, 19); from deformation-programmed material

and soft-robot design, where the local material or actuator rule controls the reachable shape space (21, 25); from inflatable and auxetic inverse design, where a desired three-dimensional target must be produced by a planar pattern or cell field rather than by a decorative surface (22–24); and from mechanical metamaterials, where local cell rules can create global deformation modes but can also produce unintended instabilities if the cell rule is not made explicit (26, 27, 31). WaveVis currently encodes those lessons as hard source gates rather than as a mature continuous optimizer.

The public app serves a static build. The architecture PDF is placed in the public proofs directory as `wavevis-system-architecture.pdf` and linked from the WaveVis control headers. The old connector proof remains a source artifact for the mechanism decision, but the visible paper action now points to this architecture record because the current work is broader than connector assignment.

17 Discussion

The main design lesson is that WaveVis cannot be corrected by tuning only the side profile. The user-facing failure was three-dimensional: side walls, swept tubes, missing full-array linkages, broken side-view semantics, pointy front fins, and zig-zag legs. The architecture therefore now uses a single curated target contract rather than a manual editor contract. The sheet generator is responsible for creating a localized breaking-wave displacement field with lateral fade, boundary preservation, topology completeness, and a visible curled lip.

This distinction also clarifies future work. If the simulator produces a good center profile but a bad side view, the profile is only evidence that the curve exists somewhere. The real artifact still fails if the full three-dimensional render hides the curl or reads as an extrusion. Similarly, if the mesh is clean but the wave looks like a low mound, the mechanism is not enough. Both the shape and the linkage must be correct.

A newer lesson is that defect naming is not progress by itself. If a rendered state shows dimples, under-overhang pockets, side walls, a pointy side surrogate, or a missing curl, the correct workflow is source repair, geometry-check update, rendered screenshot verification, and public-bundle verification. The current checks name this directly through `noVisibleDimplePocket`, `readableSideProfileCavityBlock`, the side terminal-backtrack floor, and the side-camera fit: the lip must remain an open downturned branch with visible clearance and a return that advances into the flat sheet instead of folding into a pucker, dark throat bite, or needle-like side notch. The same

source-of-truth rule applies to URL state. Several correction screenshots used URLs that carried
explicit height, overhang, width, lipDip, and smoothing values; ignoring those parameters caused the
page to render a different geometry than the requested test state. The startup parser is now part of
validation through `startupUrlParamCoverage`. The architecture record is useful only if it changes
the generator, state loader, and acceptance loop; it cannot substitute for a corrected human-facing
WaveVis render.

18 Acknowledgements

This record was written to reduce repeated handoff burden during WaveVis development. The repeated
complaints about side walls, missing cells, zig-zag legs, profile-view confusion, and swept-barrel
geometry are treated as acceptance criteria rather than optional polish notes.

19 Funding

No external funding is claimed in this system architecture record.

20 Author contributions

Alan N. Pham defined the target artifact, accepted and rejected visual states, and the mechanism
constraints. Codex generated this architecture record from the current WaveVis source and the active
development contract.

21 Competing interests

The author declares no competing interests for this internal architecture record.

22 Data availability

The source data for this record is the WaveVis source tree, the accepted June 24 gridded reference
family, the June 28 stage/cross-section reference, the July 1 supplied side-view line, the retained
ocean-curl references copied into the proof figure set, the local proof diagrams, rejected-candidate
verification folders, and the current simulator behavior. The rejected trace-overlay line is recorded only
as a failure class, not as accepted source data. The public PDF is served with the WaveVis static app.

23 Code availability

The relevant implementation files are `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, and `scripts/check-inverse-sheet.cjs` in the WaveVis app repository.

24 Additional information

This document supersedes the old header link to the narrow connector-contact constraint proof as the primary public WaveVis PDF. The connector proof remains an implementation reference for mechanism feasibility, but the current public artifact needs the broader system architecture and outcome-verification contract.

25 References

1. D. Terzopoulos, J. Platt, A. Barr, K. Fleischer, Elastically Deformable Models. *ACM SIGGRAPH Computer Graphics* **21**, 205–214 (1987).
2. D. Baraff, A. Witkin, presented at the Proceedings of SIGGRAPH 1998, pp. 43–54.
3. E. Grinspun, A. N. Hirani, M. Desbrun, P. Schröder, presented at the Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, pp. 62–67.
4. M. Bergou, M. Wardetzky, S. Robinson, B. Audoly, E. Grinspun, Discrete Elastic Rods. *ACM Transactions on Graphics* **27**, 63:1–63:12 (2008).
5. T. Liu, A. W. Bargteil, J. F. O’Brien, L. Kavan, Fast Simulation of Mass-Spring Systems. *ACM Transactions on Graphics* **32**, 214:1–214:7 (2013).
6. A. Witkin, M. Kass, presented at the Proceedings of SIGGRAPH 1988, pp. 159–168.
7. M. Müller, B. Heidelberger, M. Hennix, J. Ratcliff, Position Based Dynamics. *Journal of Visual Communication and Image Representation* **18**, 109–118 (2007).
8. O. Sorkine, M. Alexa, presented at the Proceedings of the Fifth Eurographics Symposium on Geometry Processing, pp. 109–116.
9. S. Bouaziz, S. Martin, T. Liu, L. Kavan, M. Pauly, Projective Dynamics: Fusing Constraint Projections for Fast Simulation. *ACM Transactions on Graphics* **33**, 154:1–154:11 (2014).

10. T. Du *et al.*, DiffPD: Differentiable Projective Dynamics. *ACM Transactions on Graphics* **41**, 13:1–13:21 (2022). 906
11. M. Skouras, B. Thomaszewski, S. Coros, B. Bickel, M. Gross, Computational Design of Actuated Deformable Characters. *ACM Transactions on Graphics* **32**, 82:1–82:10 (2013). 908
12. S. Coros *et al.*, Computational Design of Mechanical Characters. *ACM Transactions on Graphics* **32**, 83:1–83:12 (2013). 909
13. B. Thomaszewski *et al.*, Computational Design of Linkage-Based Characters. *ACM Transactions on Graphics* **33**, 64:1–64:9 (2014). 910
- 914 14. V. Megaro *et al.*, A Computational Design Tool for Compliant Mechanisms. *ACM Transactions* 915
on Graphics **36**, 82:1–82:12 (2017). 916
- 917 15. L. Zhu *et al.*, LinkEdit: Interactive Linkage Editing Using Symbolic Kinematics. *ACM Transac-* 918
tions on Graphics **34**, 99:1–99:8 (2015). 919
- 920 16. J. M. McCarthy, *Geometric Design of Linkages* (Springer, 2000). 921
- 922 17. K. H. Hunt, *Kinematic Geometry of Mechanisms* (Oxford University Press, 1978). 923
- 924 18. M. P. Bendsøe, N. Kikuchi, Generating Optimal Topologies in Structural Design Using a 925
Homogenization Method. *Computer Methods in Applied Mechanics and Engineering* **71**, 197– 926
224 (1988). 927
- 928 19. O. Sigmund, On the Design of Compliant Mechanisms Using Topology Optimization. *Mechanics* 929
of Structures and Machines **25**, 493–524 (1997). 930
- 931 20. O. Sigmund, K. Maute, Topology Optimization Approaches: A Comparative Review. *Structural* 932
and Multidisciplinary Optimization **48**, 1031–1055 (2013). 933
- 934 21. B. Bickel *et al.*, Design and Fabrication of Materials with Desired Deformation Behavior. *ACM*
Transactions on Graphics **29**, 63:1–63:10 (2010). 934
22. M. Skouras *et al.*, Designing Inflatable Structures. *ACM Transactions on Graphics* **33**, 63:1–63:10 (2014).
23. M. Konaković-Luković *et al.*, Beyond Developable: Computational Design and Fabrication with Auxetic Materials. *ACM Transactions on Graphics* **35**, 89:1–89:11 (2016).
24. J. Panetta *et al.*, Computational Inverse Design of Surface-Based Inflatables. *ACM Transactions on Graphics* **40**, 40:1–40:14 (2021).

-
- 935 25. D. Rus, M. T. Tolley, Design, Fabrication and Control of Soft Robots. *Nature* **521**, 467–475
 936 (2015).
- 937 26. C. Coullais, E. Teomy, K. de Reus, Y. Shokef, M. van Hecke, Combinatorial Design of Textured
 938 Mechanical Metamaterials. *Nature* **535**, 529–532 (2016).
- 939 27. M. Kadic, G. W. Milton, M. van Hecke, M. Wegener, 3D Metamaterials. *Nature Reviews Physics*
 940 **1**, 198–210 (2019).
- 941 28. A. N. Pham, *WaveVis overhang connector assignment: evidence rejecting all-four-equal X-cells*,
 942 Internal WaveVis constraint proof and simulator evidence figures, 2026.
29. A. N. Pham, *WaveVis inverse-sheet simulator source tree*, Local source files: src/latticeGeometry.ts, 943
 src/LatticeViewer3D.tsx, src/TargetShapeControls.tsx, scripts/check-inverse-sheet.cjs, 2026. 944
30. O. Sigmund, A 99 Line Topology Optimization Code Written in Matlab. *Structural and* 945
Multidisciplinary Optimization **21**, 120–127 (2001). 946
31. K. Bertoldi, P. M. Reis, S. Willshaw, T. Mullin, Negative Poisson’s Ratio Behavior Induced by 947
 an Elastic Instability. *Advanced Materials* **22**, 361–366 (2010). 948